RESULTS

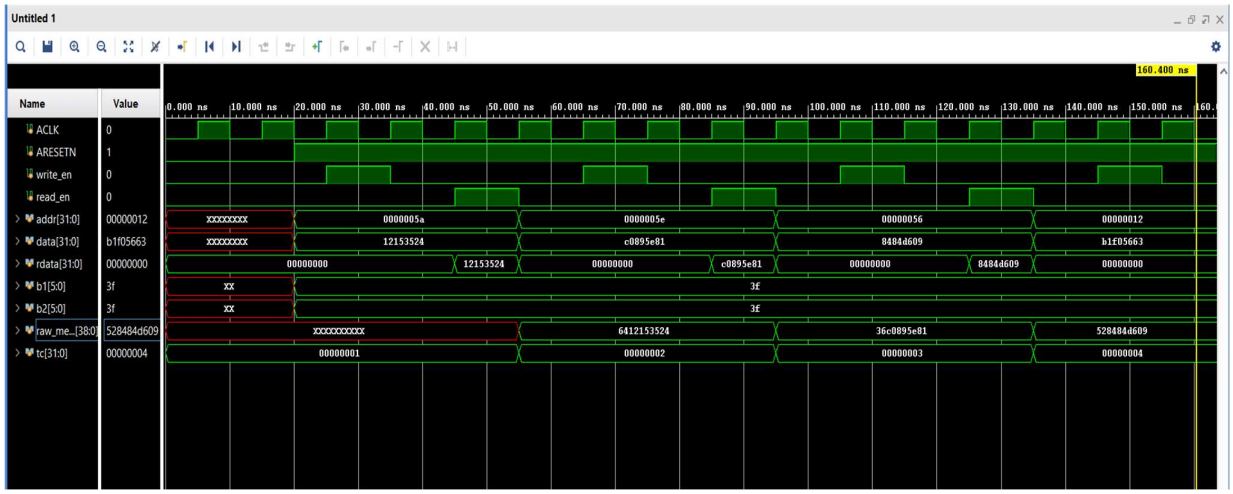


Figure 1: Simulation result of Test cases 1 to 4

Figure 1 shows the simulation results of test cases 1 to 4 under no error condition. The waveform illustrates correct AXI read and write operations with data being accurately stored and retrieved.The ECC module detects no errors, confirming proper system functionality.

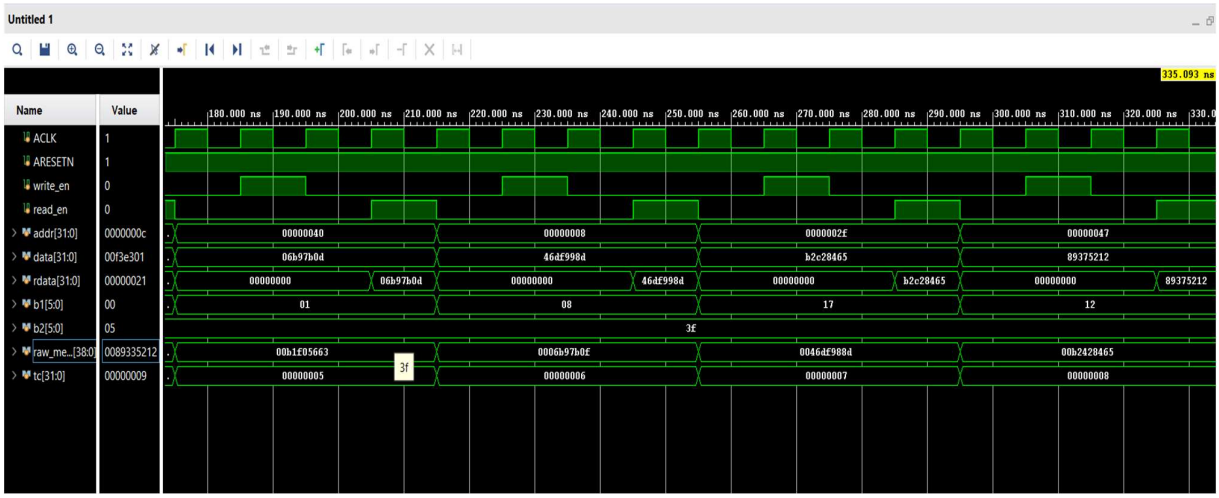


Figure 2: Simulation result of Test cases 5 to 8

As shown in Figure 2, the AXI memory interface performs read and write operations with single-bit error injection. The ECC module successfully detects and corrects the single-bit error during the read operation. The output data is restored to its original value, confirming correct SECDED operation.

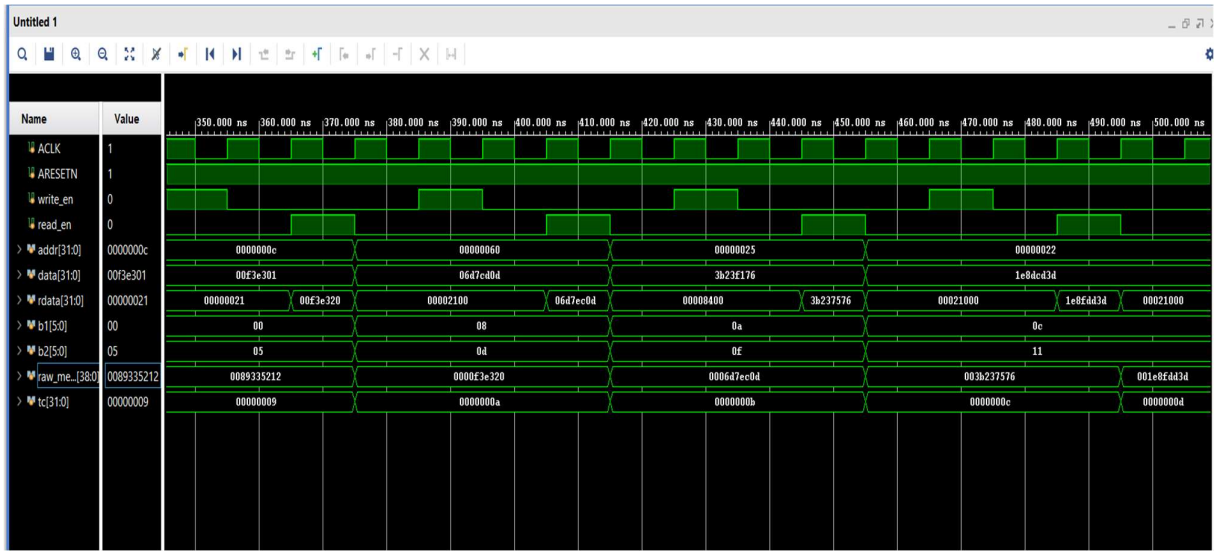


Figure 3: Simulation result of Test cases 9 to 12

As shown in Figure 3, the AXI memory interface performs read and write operations with double-bit error injection. The ECC module detects the presence of double-bit errors but cannot correct them. Hence, the corrupted data is forwarded as it is, indicating proper double-error detection functionality.

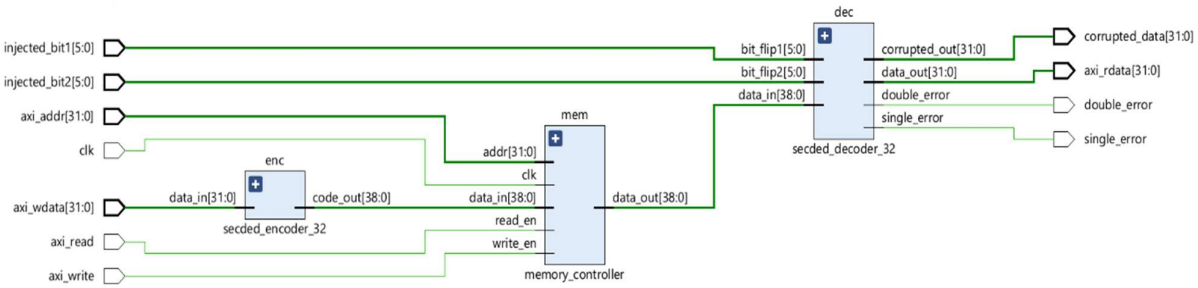


Figure 4: RTL Schematic of AXI Memory Interface with ECC

Figure 4 shows the RTL schematic of the AXI Memory Interface with ECC. It includes the encoder, memory, and decoder blocks connected with the AXI signals. The design helps in detecting and correcting errors during data transfer

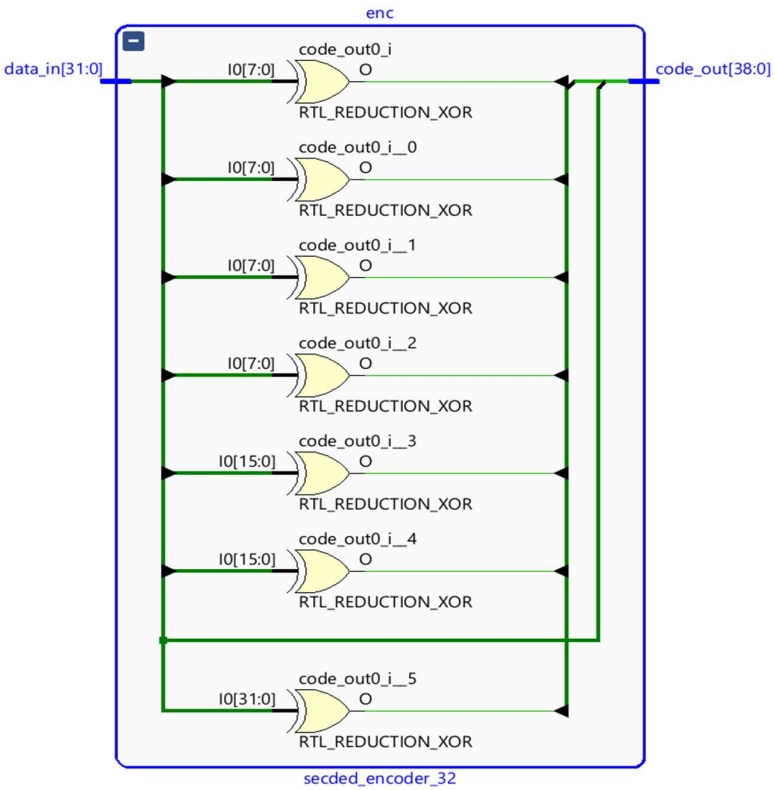


Figure 5: RTL Schematic of ECC Encoder

Figure 5 shows the RTL schematic of the ECC encoder. It generates parity bits using XOR operations on the input data bits. These parity bits are combined with the original data to form the encoded output for error detection and correction.

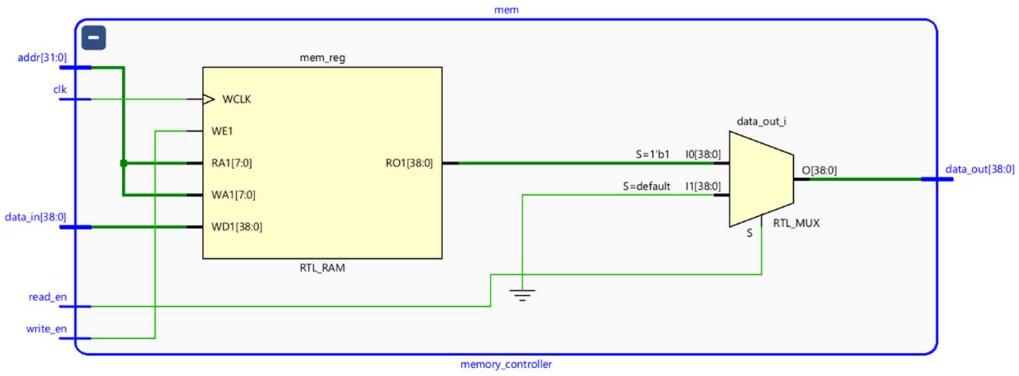


Figure 6: RTL Schematic of Memory Controller

Figure 6 shows the RTL schematic of the memory controller. It stores encoded data into RAM during write operations and retrieves it during read operations. The control signals manage proper data flow between memory and output.

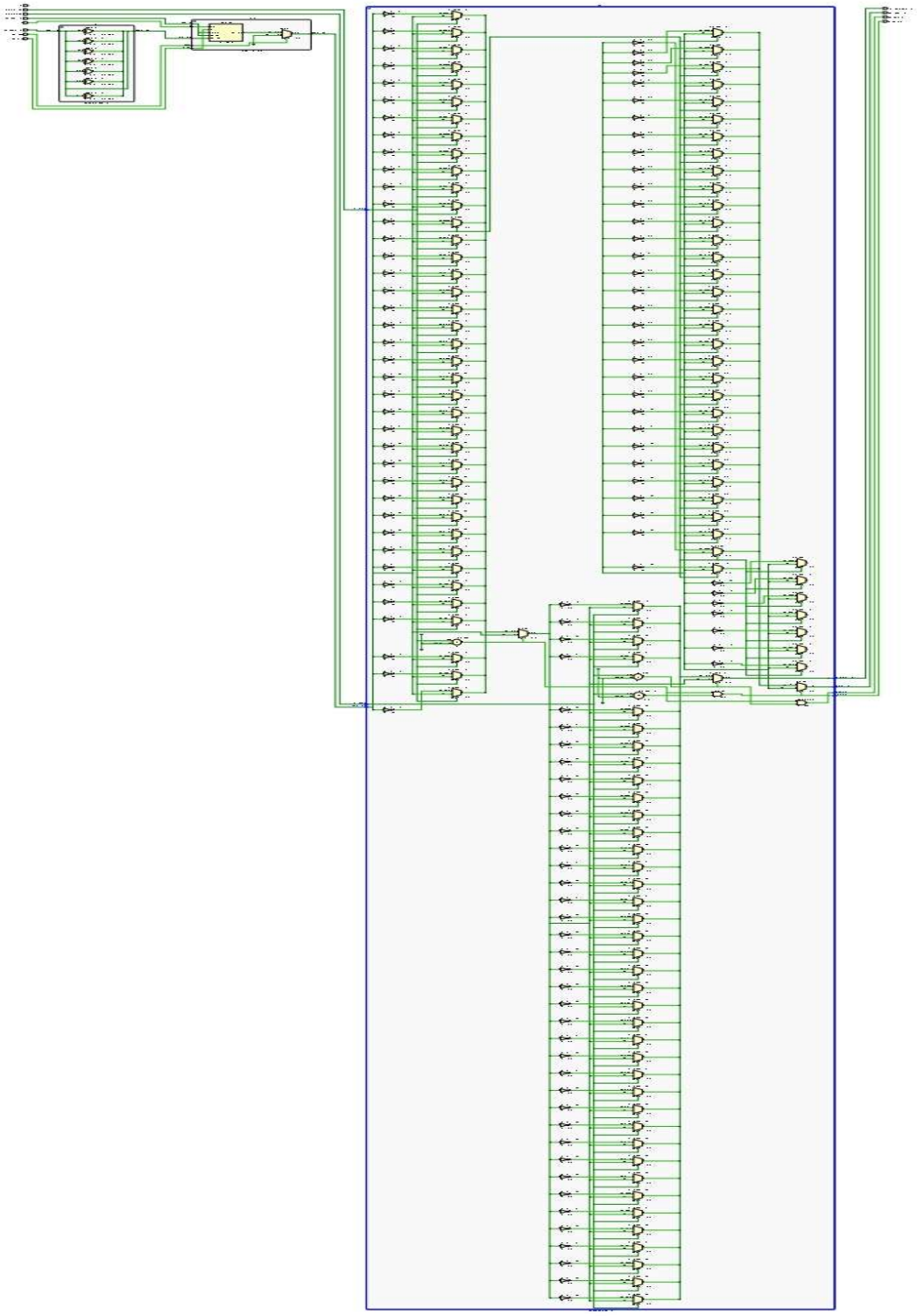


Figure 7: RTL Schematic of ECC Decoder

Figure 7 shows the RTL schematic of the ECC decoder. It checks the received data using parity bits to detect errors. The decoder corrects single-bit errors and indicates double-bit errors during read operations.

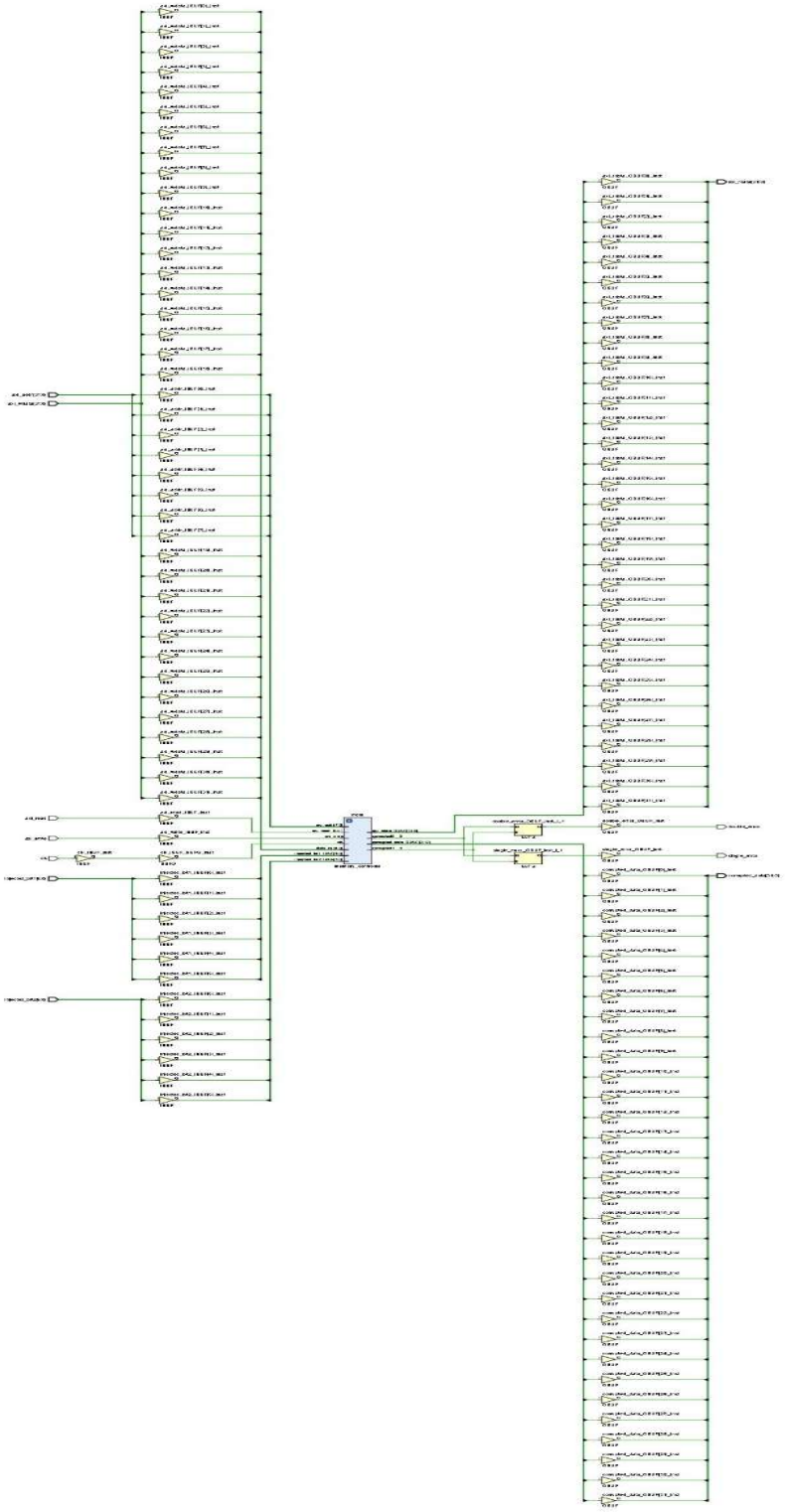


Figure 8: Synthesized Hardware Design of AXI Memory Interface with ECC

Figure 8 shows the synthesized hardware design of the AXI Memory Interface with ECC. It is the gate-level view of the complete system after synthesis. All modules work together to support memory operations with error detection and correction.

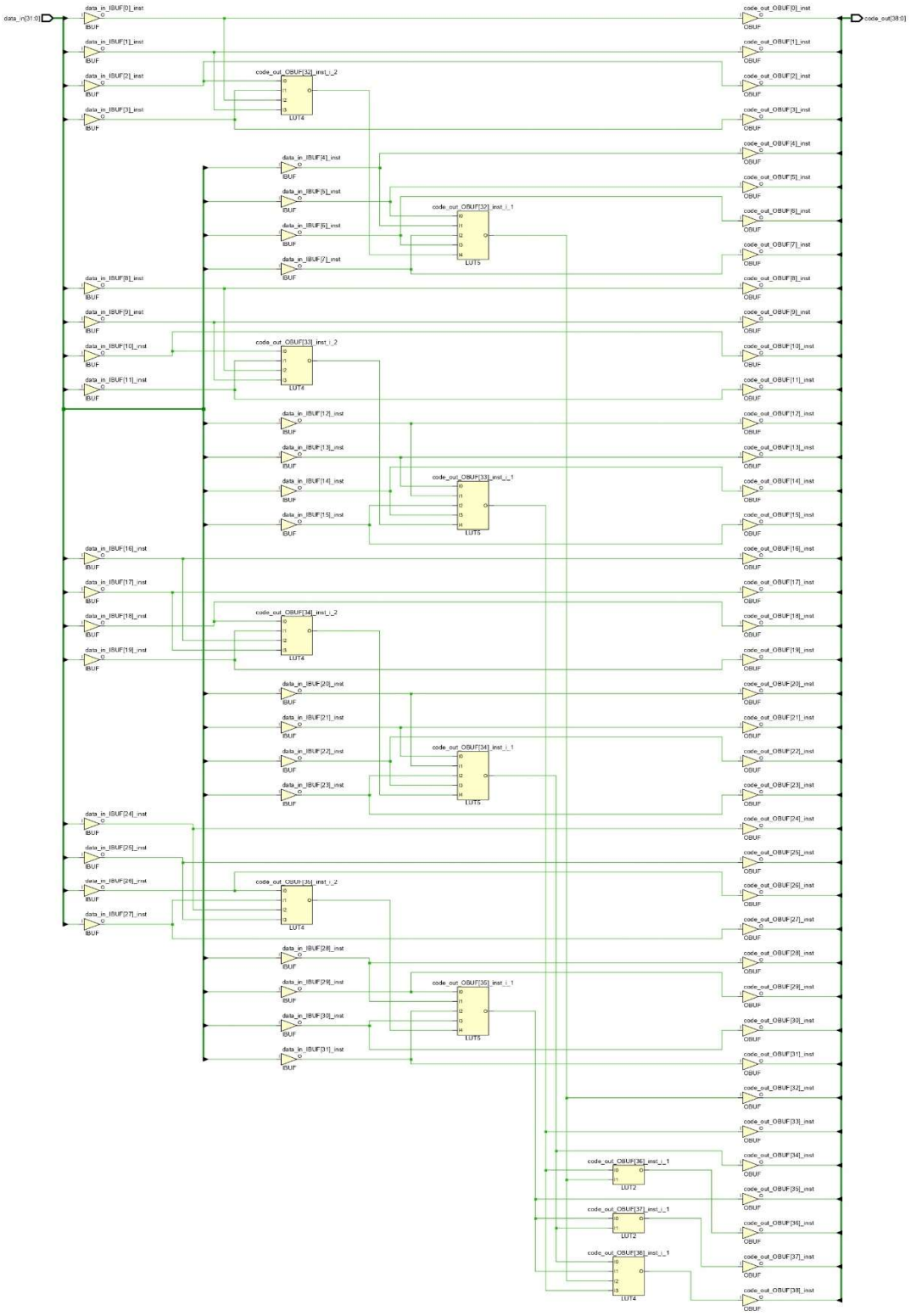


Figure 9: Synthesized Hardware Design of ECC Encoder

Figure 9 shows the synthesized hardware design of the ECC encoder. It represents the gate-level implementation using logic elements like LUTs and buffers. The encoder generates parity bits from input data for error detection and correction.

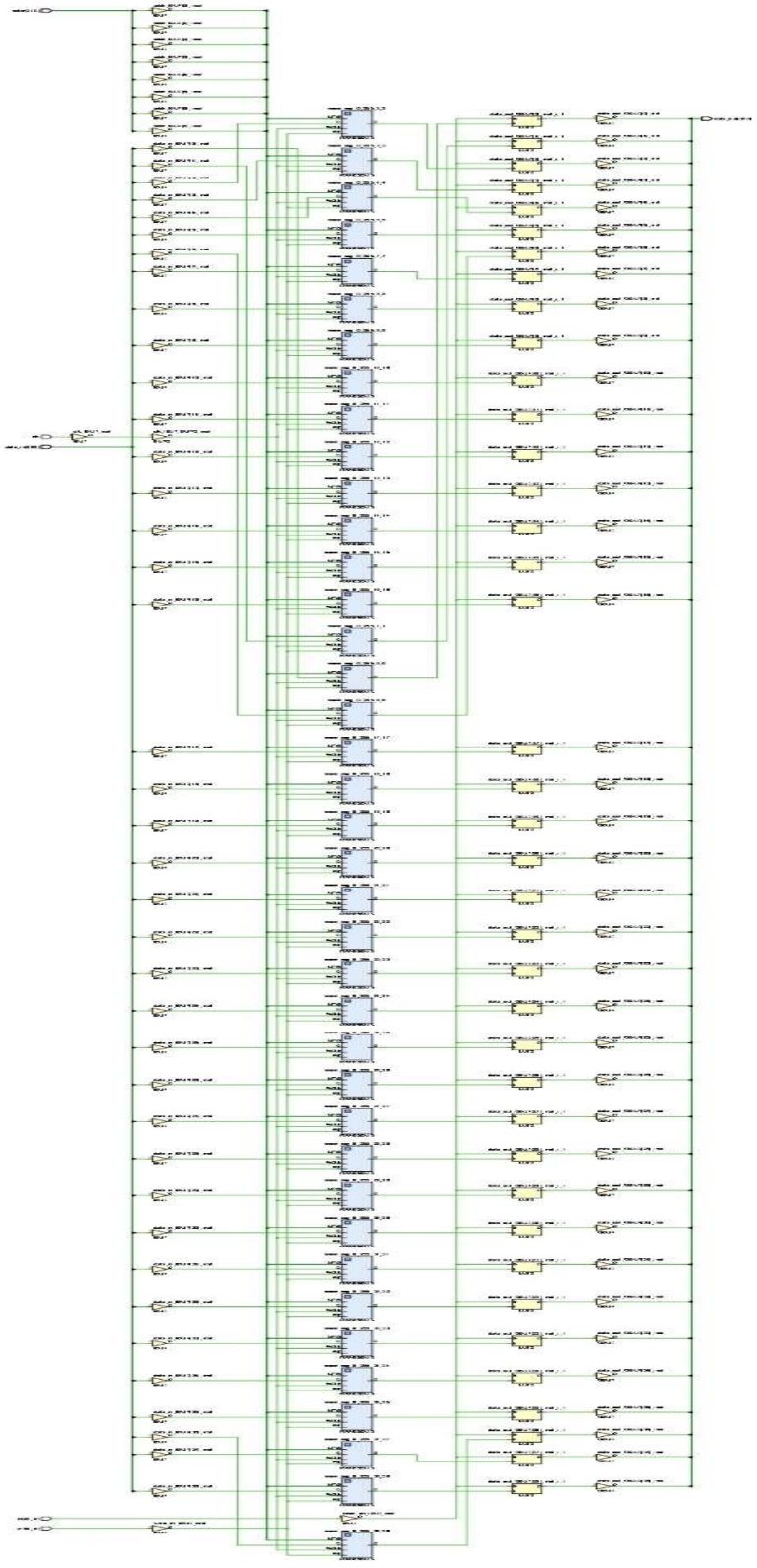


Figure 10: Synthesized Hardware Design of Memory

Figure 10 shows the synthesized hardware design of the memory block. It represents the gate-level implementation of storage elements used to store encoded data. The memory works with control signals to perform read and write operations.

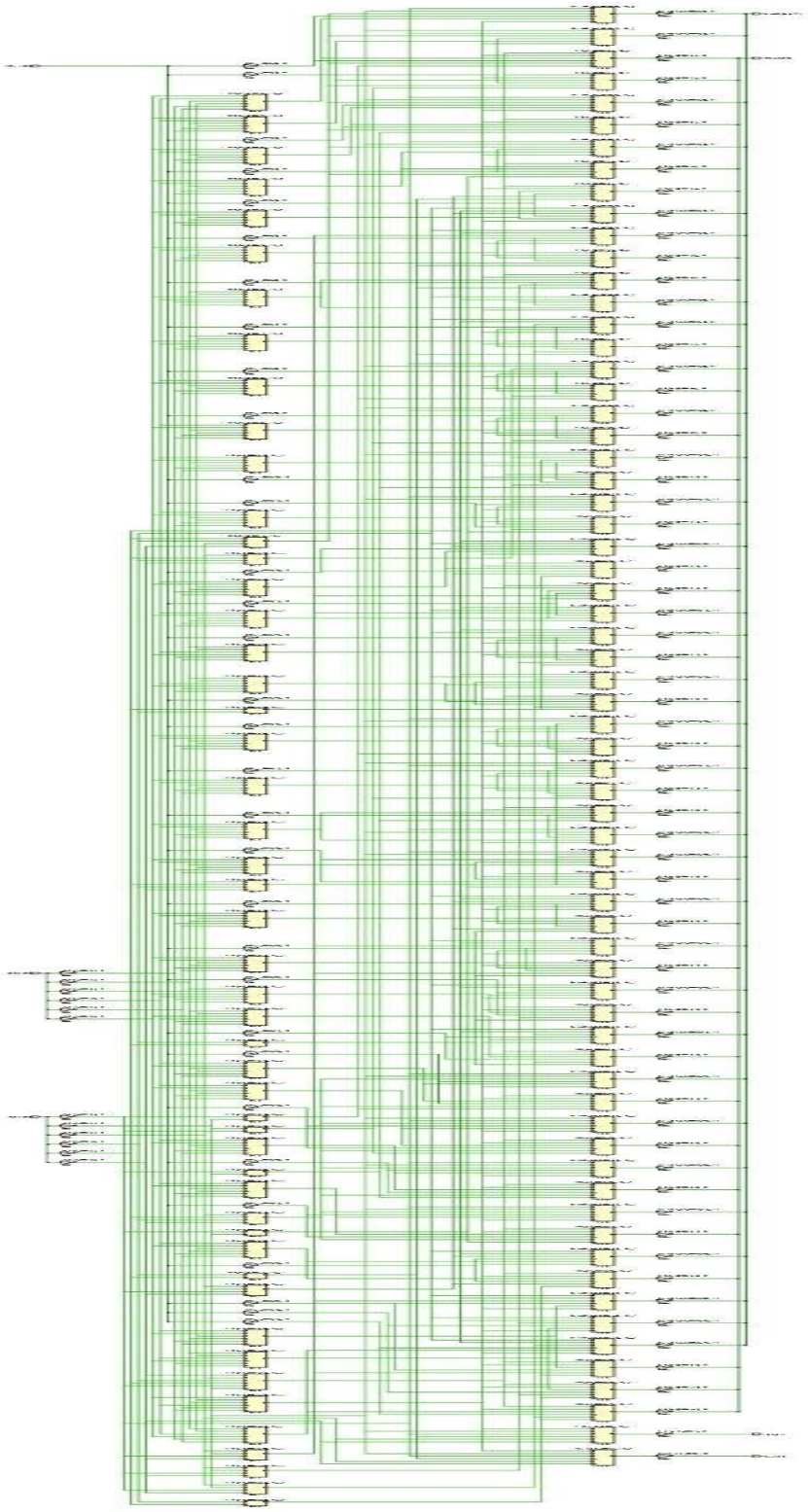


Figure 11: Synthesized Hardware Design of ECC Decoder

Figure 11 shows the synthesized hardware design of the ECC decoder. It represents the gate-level implementation used to detect and correct errors in the data. The decoder processes the received data and indicates single or double-bit errors.

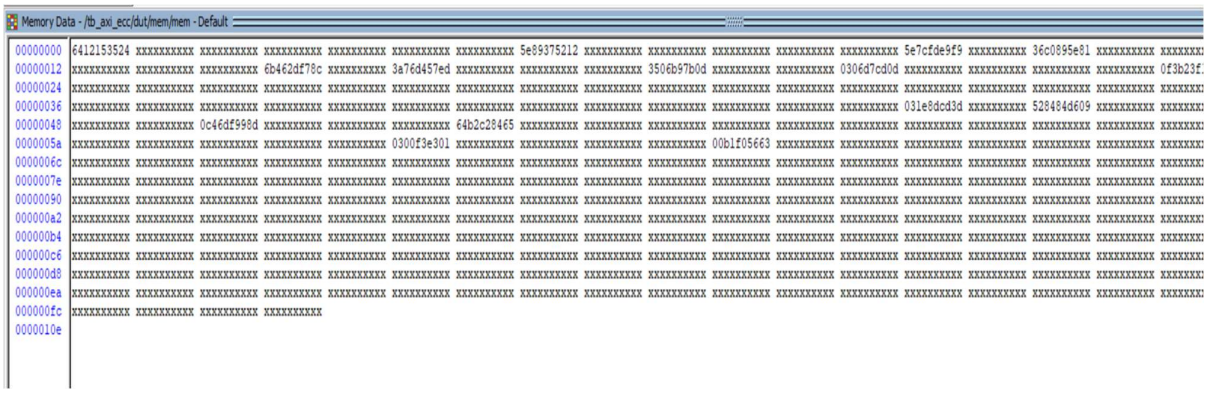


Figure 12: Memory Data View in Simulation

Figure 12 shows the memory data view in simulation. It presents the stored data values at different memory addresses. This confirms that data is written and stored correctly in memory.

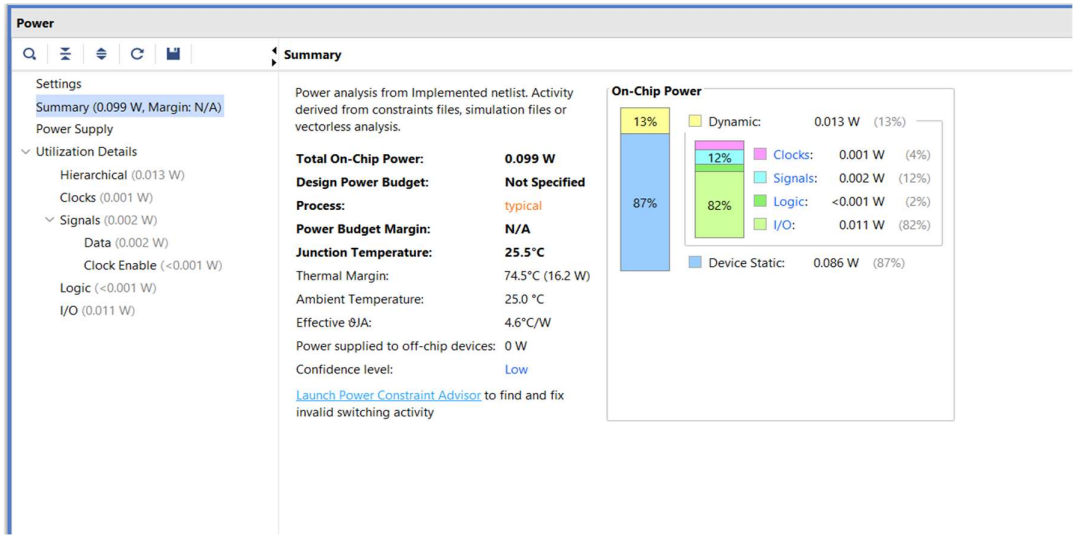


Figure 13: Power Analysis Report of AXI Memory Interface with ECC

Figure 13 presents the power analysis of the AXI Memory Interface with ECC. It shows the total, dynamic, and static power values. The results indicate that the design has low power consumpti

Name	Slice LUTs (63400)	F7 Muxes (31700)	F8 Muxes (15850)	Slice (15850)	LUT as Logic (63400)	LUT as Memory (19000)	Bonded IOB (210)	BUFGCTRL (32)
top_axi_slave_wrapper	265	64	32	146	137	128	121	1
mem (memory_controller)	263	64	32	144	135	128	0	0

Figure 14: Utilization Summary of AXI Slave Wrapper Design

This figure presents the post-synthesis resource utilization of the AXI slave wrapper and memory controller modules in Vivado. It shows the number of Slice LUTs, Flip-Flops, MUX resources, and I/O blocks used by the design. The AXI slave wrapper consumes slightly more resources due to interface control logic, while the memory controller contributes mainly to LUT usage. Overall utilization is low, indicating an area-efficient implementation suitable for FPGA deployment.